

Министерство науки и высшего образования Российской Федерации
Пензенской государственный университет
Кафедра «Вычислительная техника»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К курсовому проектированию
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Реализация алгоритма Форда-Беллмана»

29.12.25
отлично
ф.с.в.

Выполнил:

Студент группы 24BVB1

Хмельков М.А.

Принял:

к.т.н., доцент Юрова О.В.

Пенза 2025

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет Вычислительной техники

Кафедра "Вычислительная техника"

"УТВЕРЖДАЮ"

Зав. кафедрой ВТ

« » сентября 20 25

ЗАДАНИЕ

на курсовое проектирование по курсу

Логика и основы алгоритмизации в инженерных задачах
Студенту Хмелькову Максиму Алексеевичу Группа 24ВВВ1
Тема проекта Реализация алгоритма Форда-Беллмана

Исходные данные (технические требования) на проектирование

Разработка алгоритмов и программного обеспечения в соответствии с данными заданием курсового проекта.

Техническая записка должна содержать:

1. Постановку задачи;
2. Теоретическую часть задания;
3. Описание алгоритма постановки задачи;
4. Пример ручного расчета задачи и вычисления (на небольшом участке работы алгоритма);
5. Описание самой программы;
6. Источники;
7. Список литературы
8. Исполнение программы
9. Результаты работы программы

Объем работы по курсу

1. Расчетная часть

Ручной расчёт работы алгоритма

2. Графическая часть

Схема алгоритма в формате блок-схем

3. Экспериментальная часть

Тестирование программы;
Результаты работы программы на тестовых
данных.

Срок выполнения проекта по разделам

- 1 Исследование теоретической части курсового проекта
- 2 Разработка алгоритмов программы
- 3 Разработка программы
- 4 Тестирование и завершение разработки программы
- 5 Оформление пояснительной записки
- 6
- 7
- 8

Дата выдачи задания "12" сентября 2025

Дата защиты проекта " " "

Руководитель Юрова О.В.

Задание получил "25" сентября 2025г.

Студент Хмельков М.А.

Содержание

Реферат	5
Введение	6
1. Постановка задачи	7
2. Теоретическая часть задания	8
3. Описание алгоритма программы	10
4. Описание программы	15
5. Тестирование	19
6. Ручной расчёт задачи	28
Заключение	31
Список литературы	32
Приложение А	33
Листинг программы	33

Реферат

Отчет 46 стр., 11 рисунков, 1 таблица, 5 источников, 1 приложение.

АЛГОРИТМ ФОРДА-БЕЛЛМАНА, ГРАФ, МАТРИЦА СМЕЖНОСТИ, КРАТЧАЙШИЙ ПУТЬ, ОТРИЦАТЕЛЬНЫЙ ЦИКЛ, РЕЛАКСАЦИЯ, ЯЗЫК C.

Цель работы – разработка программы на языке C, реализующей алгоритм Форда-Беллмана для нахождения кратчайших путей от одной вершины до всех остальных во взвешенном ориентированном графе, с возможностью работы с отрицательными весами и обнаружением циклов отрицательного веса.

В программе предусмотрены различные способы задания графа: автоматическая генерация, ручной ввод и загрузка из файла. Реализован интерактивный консольный интерфейс, позволяющий выполнять алгоритм для выбранной стартовой вершины, выводить результаты в наглядном виде и сохранять их в файл.

Введение

Алгоритм Форда-Беллмана (англ. Bellman–Ford algorithm) является классическим алгоритмом нахождения кратчайших путей от одной вершины (источника) до всех остальных вершин во взвешенном графе. В отличие от алгоритма Дейкстры, алгоритм Форда-Беллмана может корректно обрабатывать рёбра с отрицательным весом, что значительно расширяет область его применения. Ключевой особенностью алгоритма является его способность обнаруживать циклы отрицательного веса, достижимые из стартовой вершины, что делает его незаменимым инструментом в задачах анализа сетей, маршрутизации и моделирования, где веса рёбер могут принимать произвольные значения.

Основной принцип работы алгоритма заключается в последовательной релаксации (ослаблении) всех рёбер графа. Алгоритм выполняет $V-1$ итерацию (где V — количество вершин), гарантируя нахождение кратчайших путей при их существовании. После этого выполняется дополнительная проверка всех рёбер: если на этой фазе удастся произвести дальнейшую релаксацию, то граф содержит цикл отрицательного веса, и кратчайшие пути для некоторых вершин не определены.

В качестве среды разработки мною была выбрана среда Microsoft Visual Studio 2022, язык программирования — C. Этот язык был выбран благодаря своей эффективности, низкоуровневому контролю над памятью и широкому использованию в системном программировании, что позволяет создавать наглядные и производительные реализации алгоритмов.

Целью данной курсовой работы является разработка программы на языке C, которая реализует расширенную версию алгоритма Форда-Беллмана. Программа предоставляет интерактивный интерфейс для работы с графом, поддерживая несколько способов его создания: автоматическая генерация с возможностью включения отрицательных весов и выбора типа графа, ручной ввод с клавиатуры и загрузка из файла. Программа также позволяет сохранять графы, визуализировать их в виде матрицы смежности и выполнять алгоритм с выводом подробных результатов, включая обнаружение отрицательных циклов. Разработанное приложение наглядно демонстрирует каждый этап алгоритма и предназначено как для учебных целей, так и для практического применения в задачах анализа графов.

1. Постановка задачи

Требуется разработать программу на языке C, реализующую алгоритм Форда-Беллмана для поиска кратчайших путей от одной вершины до всех остальных во взвешенном графе.

Программа должна предоставлять пользователю интерактивное меню для работы с графом, включая следующие возможности: создание графа различными способами: автоматическая генерация случайного графа с настройкой параметров (ориентированный/неориентированный, наличие отрицательных весов), ручной ввод матрицы смежности с клавиатуры, загрузка из текстового файла, отображение текущего графа в виде читаемой матрицы смежности, выполнение алгоритма Форда-Беллмана для заданной пользователем стартовой вершины, автоматическое определение и вывод информации о наличии в графе циклов отрицательного веса, достижимых из стартовой вершины, сохранение созданного графа в файл для последующего использования, сохранение результатов работы алгоритма (кратчайшие расстояния или сообщение об отрицательном цикле) в отдельный текстовый файл.

При генерации должны учитываться граничные условия, включая возможность наличия рёбер с отрицательным весом. Программа должна корректно обрабатывать все возможные сценарии: графы с отрицательными циклами, недостижимые вершины, различные конфигурации рёбер. После ввода стартовой вершины программа должна выполнить алгоритм и вывести итоговые кратчайшие расстояния или сообщение об обнаружении цикла отрицательного веса.

Устройство ввода — клавиатура.

Задание выполняется в соответствии с вариантом №14.

2. Теоретическая часть задания

Алгоритм Беллмана-Форда (Bellman-Ford algorithm) позволяет решить задачу о кратчайшем пути из одной вершины в общем случае, когда вес каждого из ребер может быть отрицательным. Для заданного взвешенного ориентированного графа $G = (V, E)$ с истоком s и весовой функцией $w : E \rightarrow \mathbb{R}$ алгоритм Беллмана-Форда возвращает логическое значение, указывающее на то, содержится ли в графе цикл с отрицательным весом, достижимый из истока. Если такой цикл существует, в алгоритме указывается, что решения не существует. Если же таких циклов нет, алгоритм выдает кратчайшие пути и их вес.

В этом алгоритме используется ослабление, в результате которого величина $d[v]$, представляющая собой оценку веса кратчайшего пути из истока s к каждой из вершин $v \in V$, уменьшается до тех пор, пока она не станет равна фактическому весу кратчайшего пути $\delta(s, v)$. Значение TRUE возвращается алгоритмом

тогда и только тогда, когда граф не содержит циклов с отрицательным весом, достижимых из истока. BELLMAN_FORD(G, w, s) 1 INITIALIZE_SINGLE_SOURCE(G, s) 2 for $i \leftarrow 1$ to $|V[G]| - 1$ 3 do for (для) каждого ребра $(u, v) \in E[G]$ 4 do RELAX(u, v, w) 5 for (для) каждого ребра $(u, v) \in E[G]$ 6 do if $d[v] > d[u] + w(u, v)$ 7 then return FALSE 8 return TRUE

На рис. 1 проиллюстрирована работа алгоритма BELLMAN_FORD с графом, содержащим 5 вершин, в котором исток находится в вершине s .

Рассмотрим, как работает алгоритм. После инициализации в строке 1 всех значений d и π , алгоритм осуществляет $|V| - 1$ проходов по ребрам графа. Каждый проход соответствует одной итерации цикла for в строках 2–4 и состоит из однократного ослабления каждого ребра графа. После $|V| - 1$ проходов в строках 5–8 проверяется наличие цикла с отрицательным весом и возвращается соответствующее булево значение. (Причины, по которым эта проверка корректно работает, станут понятными несколько позже.) В примере, приведенном на рис. 1, алгоритм Беллман-Форд возвращает значение TRUE.

На рис. 1 в вершинах графа показаны значения атрибутов d на каждом этапе работы алгоритма, а выделенные ребра указывают на значения предшественников: если ребро (u, v) выделено, то $\pi[v] = u$. В рассматриваемом примере при каждом проходе ребра ослабляются в следующем порядке: (t, x) , (t, y) , (t, z) , (x, t) , (y, x) , (y, z) , (z, x) , (z, s) , (s, t) , (s, y) . В части а рисунка показана ситуация, сложившаяся непосредственно перед первым проходом по ребрам. В частях б–д проиллюстрирована ситуация после каждого очередного

прохода по ребрам. Значения атрибутов d и π , приведенные в части д, являются окончательными.

Алгоритм Беллмана-Форда завершает свою работу в течение времени $O(V E)$, поскольку инициализация в строке 1 занимает время $\Theta(V)$, на каждый из $|V| - 1$ проходов по ребрам в строках 2–4 требуется время $\Theta(E)$, а на выполнение цикла `for` в строках 5–7 — время $O(E)$.

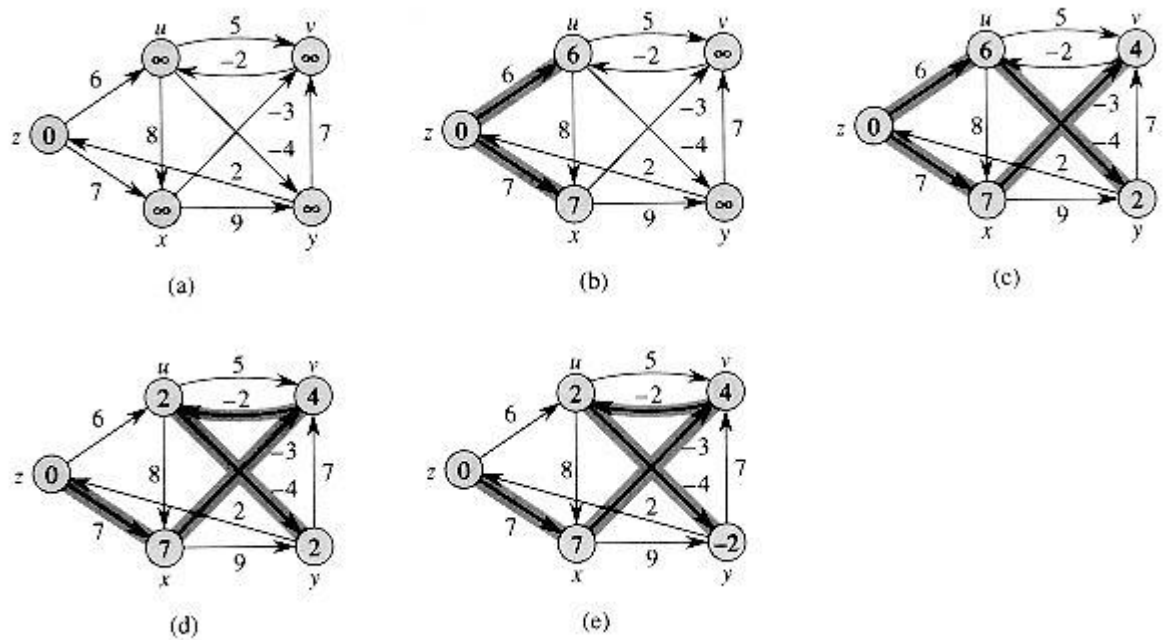


Рисунок 1- графы Форда

3. Описание алгоритма программы

Для программной реализации расширенной версии алгоритма Форда-Беллмана потребуется использовать динамические структуры данных и модульную архитектуру. Основные компоненты программы включают: динамическую матрицу смежности для хранения графа, массив расстояний, а также набор функций для управления жизненным циклом графа и выполнения самого алгоритма.

Итак, граф представляется в виде квадратной матрицы `matrix` типа `int**`, размер которой определяется во время выполнения программы. Это позволяет работать с графами произвольного размера в пределах доступной памяти. Каждый элемент `matrix[i][j]` содержит вес ребра из вершины `i` в вершину `j`. Значение 0 обозначает отсутствие непосредственного ребра между этими вершинами.

Алгоритм работы программы следует следующей последовательности:

1. Создание/Загрузка графа. Пользователю предоставляется выбор: сгенерировать случайный граф, ввести его вручную или загрузить из файла. При генерации можно задать тип графа (ориентированный/неориентированный) и разрешить наличие отрицательных весов. Функции `createMatrix`, `generateRandomGraph`, `inputGraphFromKeyboard` и `loadGraphFromFile` отвечают за формирование матрицы смежности.

2. Инициализация алгоритма. После выбора стартовой вершины `start` в функции `bellmanFord` создаётся массив `distance`. Все его элементы инициализируются значением `INF` (очень большое число), кроме `distance[start]`, которое устанавливается равным 0.

3. Фаза релаксации. Выполняется основной цикл, повторяющийся `size - 1` раз (где `size` — количество вершин). На каждой итерации проверяются все возможные пары вершин (`from`, `to`). Если между ними существует ребро (`matrix[from][to] != 0`), выполняется попытка релаксации: если текущее расстояние до вершины `from` не бесконечно и сумма `distance[from] + вес_ребра` меньше текущего расстояния до `to`, то расстояние `distance[to]` обновляется.

4. Проверка на отрицательные циклы. После завершения `size - 1` итераций осуществляется дополнительный проход по всем рёбрам графа. Если на этом этапе обнаруживается возможность выполнить релаксацию, это однозначно свидетельствует о наличии в графе цикла отрицательного веса, достижимого из стартовой вершины.

5. Вывод результатов. Программа выводит на экран и сохраняет в файл `result.txt` итоговые расстояния от стартовой вершины до всех остальных. В случае обнаружения отрицательного цикла выводится соответствующее предупреждение о том, что кратчайшие пути не существуют.

Для наглядности и удобства пользователя программа реализует интерактивное консольное меню, позволяющее последовательно выполнять все этапы: создание графа, его просмотр, запуск алгоритма и сохранение данных.

Ниже представлен псевдокод ключевых функций `bellmanFord()` и частично `main()`, отражающий логику работы программы.

bellmanFord()

выделить память под массив `distance` размером `size`

для `i=0` пока `i<size` делать `i=i+1`

`distance[i] = INF`

`distance[start] = 0`

вывести "=== ВЫПОЛНЕНИЕ АЛГОРИТМА ==="

для `iteration=1` пока `iteration<=size-1` делать `iteration=iteration+1`

`changes = 0`

для `from=0` пока `from<size` делать `from=from+1`

для `to=0` пока `to<size` делать `to=to+1`

если `matrix[from][to] != 0`

если `distance[from] != INF` и `distance[from] + matrix[from][to] < distance[to]`

`distance[to] = distance[from] + matrix[from][to]`

`changes = 1`

если `changes == 0`

прервать цикл

`hasNegativeCycle = 0`

для `from=0` пока `from<size` делать `from=from+1`

для `to=0` пока `to<size` делать `to=to+1`

```

если matrix[from][to] != 0

    если distance[from] != INF и distance[from] + matrix[from][to] < distance[to]

        hasNegativeCycle = 1

    прервать циклы

если hasNegativeCycle == 1 прервать цикл

вывести "=== РЕЗУЛЬТАТЫ ==="

вывести "Стартовая вершина: start"

если hasNegativeCycle == 1

    вывести "Граф содержит отрицательный цикл!"

    вывести "Кратчайшие пути не существуют!"

для i=0 пока i<size делать i=i+1

    вывести "До вершины i: "

    если distance[i] == INF

        вывести "нет пути"

    иначе если i == start

        вывести "0 (стартовая вершина)"

    иначе

        вывести distance[i]

сохранить все результаты в файл "result.txt"

освободить память массива

distance_tmain()

инициализировать генератор случайных чисел

```

установить русскую локаль

вывести заголовок курсовой работы

пока истина

вывести меню с вариантами 1-5

считать выбор пользователя choice

если choice == 1

вывести подменю создания графа (генерация, ручной ввод, загрузка из файла)

считать выбор graphChoice

если graphChoice == 1

запросить size (количество вершин)

запросить тип графа (ориентированный/неориентированный)

запросить наличие отрицательных весов

вызвать generateRandomGraph

иначе если graphChoice == 2

запросить size

запросить тип графа

вызвать inputGraphFromKeyboard

иначе если graphChoice == 3

запросить имя файла

вызвать loadGraphFromFile

иначе если choice == 2

если граф создан, вызвать printMatrix, иначе вывести предупреждение

иначе если choice == 3

если граф создан

запросить номер стартовой вершины start


```
        если start корректен, вызвать bellmanFord  
  
    иначе если choice == 4  
  
        если граф создан, запросить имя файла и вызвать saveGraphToFile  
  
    иначе если choice == 5  
  
        освободить память (если граф создан)  
  
        вывести сообщение о завершении  
  
        завершить программу  
  
    иначе  
  
        вывести "Неверный выбор!"
```

4. Описание программы

Для написания данной программы использован язык программирования С. Язык программирования С является универсальным процедурным языком, который сочетает в себе свойства языков высокого и низкого уровней, что обеспечивает эффективность выполнения и полный контроль над ресурсами системы, что важно для реализации алгоритмов обработки графов с динамическими структурами данных.

Проект был создан в виде консольного приложения Win32 (Visual C++). Программа является многомодульной и состоит из нескольких функций: `main`, `bellmanFord`, `createMatrix`, `freeMatrix`, `printMatrix`, `generateRandomGraph`, `inputGraphFromKeyboard`, `loadGraphFromFile`, `saveGraphToFile`, `getNumber` и других вспомогательных.

Работа программы начинается с вывода на экран заголовка курсовой работы и данных студента. Затем программа переходит в цикл главного меню, где пользователь может выбрать одно из пяти действий.

Основной функционал программы разбит на следующие этапы:

1. Создание и загрузка графа

При выборе первого пункта меню программа предлагает три способа задания графа:

- 1) Автоматическая генерация случайного графа
- 2) Ручной ввод матрицы смежности с клавиатуры
- 3) Загрузка графа из текстового файла

Для автоматической генерации используется функция `generateRandomGraph`:

```
int** generateRandomGraph(int size, int includeNegative, int isDirected) {  
    int** matrix = createMatrix(size);  
    srand(time(NULL));  
  
    if (isDirected) {  
        for (int i = 0; i < size; i++) {  
            for (int j = 0; j < size; j++) {  
                if (i != j) {  
                    if (rand() % 100 < 50) {  
                        if (includeNegative) {  
                            matrix[i][j] = (rand() % 121) - 20;  
                        } else {  
                            matrix[i][j] = rand() % 100 + 1;  
                        }  
                    }  
                }  
            }  
        }  
    }  
}
```

```

        }
    }
}
}
}
return matrix;
}

```

2. Просмотр графа

Функция `printMatrix` выводит текущий граф в виде форматированной матрицы смежности, где:

- 1) Значение 0 на главной диагонали обозначает отсутствие петли
- 2) Значение 0 вне главной диагонали отображается как точка (.), обозначая отсутствие ребра
- 3) Ненулевые значения отображаются как веса рёбер

3. Выполнение алгоритма Форда-Беллмана

При выборе этого пункта программа запрашивает номер стартовой вершины и вызывает функцию `bellmanFord`, которая реализует алгоритм:

```

void bellmanFord(int** matrix, int size, int start) {
    int* distance = (int*)malloc(size * sizeof(int));
    int hasNegativeCycle = 0;

    for (int i = 0; i < size; i++) {
        distance[i] = INF;
    }
    distance[start] = 0;

    for (int iteration = 1; iteration <= size - 1; iteration++) {
        int changes = 0;
        for (int from = 0; from < size; from++) {
            for (int to = 0; to < size; to++) {
                if (matrix[from][to] != 0) {
                    if (distance[from] != INF &&
                        distance[from] + matrix[from][to] < distance[to]) {

```

```

        distance[to] = distance[from] + matrix[from][to];
        changes = 1;
    }
}
}
}
if (changes == 0) {
    break;
}
}

```

```

for (int from = 0; from < size; from++) {
    for (int to = 0; to < size; to++) {
        if (matrix[from][to] != 0) {
            if (distance[from] != INF &&
                distance[from] + matrix[from][to] < distance[to]) {
                hasNegativeCycle = 1;
                break;
            }
        }
    }
    if (hasNegativeCycle) break;
}

```

```

printf("\n=== РЕЗУЛЬТАТЫ ===\n");
if (hasNegativeCycle) {
    printf("Граф содержит отрицательный цикл!\n");
    printf("Кратчайшие пути не существуют!\n\n");
}

```

```

for (int i = 0; i < size; i++) {
    printf("До вершины %d: ", i);
    if (distance[i] == INF) {
        printf("нет пути\n");
    } else if (i == start) {

```

```

        printf("0 (стартовая вершина)\n");
    } else {
        printf("%d\n", distance[i]);
    }
}

FILE* file = fopen("result.txt", "w");
if (file) {
    fclose(file);
    printf("\nРезультаты сохранены в файл 'result.txt'\n");
}

free(distance);
}

```

4. Сохранение графа

Программа позволяет сохранить текущий граф в текстовый файл с помощью функции `saveGraphToFile`, которая записывает размер графа и всю матрицу смежности.

5. Выход из программы

При завершении работы программа освобождает всю выделенную динамическую память с помощью функции `freeMatrix` и выводит сообщение о завершении.

Программа включает механизмы обработки ошибок:

- 1) Проверка корректности ввода данных
- 2) Контроль за выделением и освобождением памяти
- 3) Обработка ошибок при работе с файлами

Пользовательский интерфейс реализован в виде интерактивного консольного меню с понятными подсказками, что делает программу удобной для использования как в учебных целях, так и для практического применения.

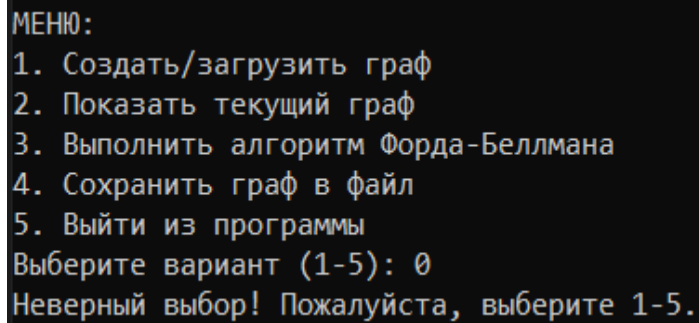
5. Тестирование

Среда разработки Microsoft Visual Studio предоставляет все средства, необходимые при разработке и отладке консольных приложений на языке C.

Тестирование проводилось в рабочем порядке, в процессе разработки, а также после завершения написания программы. В ходе тестирования было выявлено и исправлено множество проблем, связанных с вводом данных, форматированием выводимой информации, логикой работы алгоритма, взаимодействием функций и управлением динамической памятью.

Ниже продемонстрированы результаты тестирования программы при различных сценариях работы:

На рисунке представлена проверка пунктов меню.



```
МЕНЮ:  
1. Создать/загрузить граф  
2. Показать текущий граф  
3. Выполнить алгоритм Форда-Беллмана  
4. Сохранить граф в файл  
5. Выйти из программы  
Выберите вариант (1-5): 0  
Неверный выбор! Пожалуйста, выберите 1-5.
```

Рисунок 2 - Проверка пунктов меню

На рисунке представлена автоматическая генерация графа.

```
МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 1

=== ВЫБЕРИТЕ СПОСОБ СОЗДАНИЯ ГРАФА ===
1. Автоматическая генерация (случайный граф)
2. Ручной ввод с клавиатуры
3. Загрузка из файла
4. Вернуться в главное меню
Выберите вариант (1-4): 1

Введите количество вершин (2-7): 5

Выберите тип графа:
1. Ориентированный граф (направленные ребра)
2. Неориентированный граф (симметричные ребра)
Выберите вариант (1-2): 1

Включить отрицательные веса?
1. Да
2. Нет
Выберите (1-2): 1

Случайный граф успешно создан!
Тип: ориентированный граф
```

Рисунок 3 - Автоматическая генерация графа

На рисунке вывели сгенерированный граф.

```
МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 2

=== МАТРИЦА СМЕЖНОСТИ ГРАФА ===

      V0  V1  V2  V3  V4
V0:    0  60  32  .  -11
V1:   28   0  .  41  45
V2:    . -17   0  .  .
V3:    .  .  .   0  .
V4:    .  -4  .  .   0
```

Рисунок 4 - Вывод графа

На рисунке представлена генерация и вывод графа с другим вариантом условий.

```
МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 1

=== ВЫБЕРИТЕ СПОСОБ СОЗДАНИЯ ГРАФА ===
1. Автоматическая генерация (случайный граф)
2. Ручной ввод с клавиатуры
3. Загрузка из файла
4. Вернуться в главное меню
Выберите вариант (1-4): 1

Введите количество вершин (2-7): 5

Выберите тип графа:
1. Ориентированный граф (направленные ребра)
2. Неориентированный граф (симметричные ребра)
Выберите вариант (1-2): 2

Включить отрицательные веса?
1. Да
2. Нет
Выберите (1-2): 2

Случайный граф успешно создан!
Тип: неориентированный граф

МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 2

=== МАТРИЦА СМЕЖНОСТИ ГРАФА ===

      V0  V1  V2  V3  V4
V0:    0  16  .   .   .
V1:   16   0  17  15  85
V2:    .  17   0   9  18
V3:    .  15   9   0  62
V4:    .  85  18  62   0
```

Рисунок 5 - Генерация и вывод графа с другим вариантом условий

На рисунке представлено ручное создание графа и вывод его на экран.

```
=== ВЫБЕРИТЕ СПОСОБ СОЗДАНИЯ ГРАФА ===
1. Автоматическая генерация (случайный граф)
2. Ручной ввод с клавиатуры
3. Загрузка из файла
4. Вернуться в главное меню
Выберите вариант (1-4): 2

Введите количество вершин (2-7): 4

Выберите тип графа:
1. Ориентированный граф (направленные ребра)
2. Неориентированный граф (симметричные ребра)
Выберите вариант (1-2): 2

Введите веса ребер (0 - если ребра нет):
(Неориентированный граф - ребра симметричные)
Вес ребра V0 <-> V1: -10
Вес ребра V0 <-> V2: 16
Вес ребра V0 <-> V3: 0
Вес ребра V1 <-> V2: 129
Вес ребра V1 <-> V3: -98
Вес ребра V2 <-> V3: 0

Граф успешно введен!
Тип: неориентированный граф

МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 2

=== МАТРИЦА СМЕЖНОСТИ ГРАФА ===

      V0  V1  V2  V3
V0:   0 -10  16   .
V1: -10   0 129 -98
V2:  16 129   0   .
V3:   . -98   .   0
```

Рисунок 6 - Ручное создание графа и вывод его в экран

На рисунке представлен алгоритм Форда-Беллмана с отрицательным циклом.

```
МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 2

=== МАТРИЦА СМЕЖНОСТИ ГРАФА ===

      V0 V1 V2 V3 V4 V5
V0:   0 -17 . 25 . 84
V1: -17  0 -4 . 28 .
V2:  . -4  0 72 . .
V3: 25  . 72  0 . 80
V4:  . 28  . .  0 .
V5: 84  . . 80 .  0

МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 3

Введите номер стартовой вершины (0-5): 0

=== ВЫПОЛНЕНИЕ АЛГОРИТМА ===

=== РЕЗУЛЬТАТЫ ===
Стартовая вершина: 0

Граф содержит отрицательный цикл!
Кратчайшие пути не существуют!

До вершины 0: 0 (стартовая вершина)
До вершины 1: -161
До вершины 2: -157
До вершины 3: -111
До вершины 4: -125
До вершины 5: -52

Результаты сохранены в файл 'result.txt'
```

Рисунок 7 - Алгоритм Форда-Беллмана с отрицательным циклом

На рисунке представлен алгоритм Форда-Беллмана без отрицательных циклов.

```
МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 2

=== МАТРИЦА СМЕЖНОСТИ ГРАФА ===

      V0  V1  V2  V3  V4  V5
V0:   0  40  49  13  16  .
V1:   .   0  65  11  96  26
V2:   .   .   0  17  .   .
V3:  12   .  95   0  89  98
V4:   .  61   9   .   0   .
V5:  60  90   .   .  87   0

МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 3

Введите номер стартовой вершины (0-5): 2

=== ВЫПОЛНЕНИЕ АЛГОРИТМА ===

=== РЕЗУЛЬТАТЫ ===
Стартовая вершина: 2

До вершины 0: 29
До вершины 1: 69
До вершины 2: 0 (стартовая вершина)
До вершины 3: 17
До вершины 4: 45
До вершины 5: 95

Результаты сохранены в файл 'result.txt'
```

Рисунок 8 - Алгоритм Форда-Беллмана без отрицательного цикла

На рисунке представлено сохранение графа в файл.

```
=== ВЫБЕРИТЕ СПОСОБ СОЗДАНИЯ ГРАФА ===
1. Автоматическая генерация (случайный граф)
2. Ручной ввод с клавиатуры
3. Загрузка из файла
4. Вернуться в главное меню
Выберите вариант (1-4): 1

Введите количество вершин (2-7): 7

Выберите тип графа:
1. Ориентированный граф (направленные ребра)
2. Неориентированный граф (симметричные ребра)
Выберите вариант (1-2): й
Ошибка! Пожалуйста, выберите 1 или 2

Выберите тип графа:
1. Ориентированный граф (направленные ребра)
2. Неориентированный граф (симметричные ребра)
Выберите вариант (1-2): 1

Включить отрицательные веса?
1. Да
2. Нет
Выберите (1-2): 1

Случайный граф успешно создан!
Тип: ориентированный граф

МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 4

Введите имя файла для сохранения (по умолчанию graph.txt):

Граф успешно сохранен в файл 'graph.txt'
```

Рисунок 9 - Сохранение графа в файл

На рисунке представлена загрузка графа из файла.

```
МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 1

=== ВЫБЕРИТЕ СПОСОБ СОЗДАНИЯ ГРАФА ===
1. Автоматическая генерация (случайный граф)
2. Ручной ввод с клавиатуры
3. Загрузка из файла
4. Вернуться в главное меню
Выберите вариант (1-4): 3

Введите имя файла: graph.txt

Граф успешно загружен из файла 'graph.txt'

МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 2

=== МАТРИЦА СМЕЖНОСТИ ГРАФА ===

      V0  V1  V2  V3  V4  V5  V6
V0:    0   .   .   .  36  -2   .
V1:   46   0  88   . 100  69  29
V2:    . -12   0   .  40 100  64
V3:    .   5   .   0 -18   .  -8
V4:    .  -1  21  17   0   .   .
V5:    .   .   .  49  32   0  32
V6:    .   .  93   . -10   .   0
```

Рисунок 10 - Загрузка графа из файла

Таблица 1 – Описание поведения программы при тестировании

Описание теста	Ожидаемый результат	Полученный результат
Запуск программы	Вывод заголовка работы, данных студента и главного меню с вариантами 1–5	Верно
Выбор пункта 1 (Создать/загрузить граф)	Появление подменю с вариантами: генерация, ручной ввод, загрузка из файла	Верно
Автоматическая генерация (ориентированный граф, без отрицательных весов)	Создание случайной матрицы смежности размером 4×4 с весами от 1 до 100	Верно
Автоматическая генерация (неориентированный граф, с отрицательными весами)	Создание симметричной матрицы с весами от –20 до 100	Верно
Ручной ввод графа (ориентированный, 4 вершины)	Возможность ввода весов для каждого ребра с клавиатуры	Верно
Загрузка графа из файла	Чтение матрицы смежности из файла и вывод на экран	Верно

Продолжение таблицы 1

Выбор пункта 2 (Показать текущий граф)	Вывод матрицы смежности в формате “0” на диагонали, “.” при отсутствии ребра, числа – веса	Верно
Выбор пункта 3 (Выполнить алгоритм) без создания графа	Сообщение: “Граф не создан!”	Верно
Выполнение алгоритма для графа с отрицательным циклом	Вывод: “Граф содержит отрицательный цикл!” и “Кратчайшие пути не существуют!”	Верно
Выполнение алгоритма для графа без отрицательного цикла	Вывод кратчайших расстояний от стартовой вершины до всех остальных	Верно
Сохранение графа в файл	Создание файла с размером матрицы и значениями весов	Верно
Сохранение результатов алгоритма	Автоматическое сохранение в result.txt с описанием расстояний или наличия цикла	Верно
Ввод некорректной стартовой вершины (например, -1 или больше размера графа)	Сообщение об ошибке и запрос повторного ввода	Верно
Ввод нечисловых данных в меню	Программа не “падает”, выводит сообщение об ошибке	Верно
Выход из программы (пункт 5)	Освобождение памяти и корректное завершение	Верно

В результате тестирования было установлено, что программа корректно обрабатывает все предусмотренные сценарии работы, правильно вычисляет кратчайшие пути, своевременно обнаруживает отрицательные циклы и обеспечивает устойчивую работу при различных входных данных. Все выявленные в процессе тестирования ошибки были исправлены до финальной версии программы.

6. Ручной расчёт задачи

Проведём ручную проверку работы алгоритма Форда–Беллмана для заданного графа.

Дано:

Ориентированный граф с 5 вершинами (V0–V4). Матрица смежности (точка означает отсутствие ребра):

	V0	V1	V2	V3	V4
V0:	0	.	.	23	42
V1:	.	0	.	25	.
V2:	.	.	0	.	.
V3:	96	68	.	0	61
V4:	.	99	18	25	0

Стартовая вершина: V4.

Для начала проинициализируем данные для вершины 4.
Записываем начальные расстояния до всех вершин:
V0: ∞ ; V1: ∞ ; V2: ∞ ; V3: ∞ ; V4: 0.

Выполняем первую итерацию релаксации. Просматриваем все рёбра графа. Улучшаем расстояния, если найден более короткий путь.

Из V4 есть рёбра:

V4 $\rightarrow$ V1 (вес 99): $V1 = \min(\infty, 0+99) = 99$
V4 $\rightarrow$ V2 (вес 18): $V2 = \min(\infty, 0+18) = 18$
V4 $\rightarrow$ V3 (вес 25): $V3 = \min(\infty, 0+25) = 25$

Остальные рёбра не дают улучшений, так как до их начальных вершин ещё ∞ .

После 1-й итерации:

V0: ∞ ; V1: 99; V2: 18; V3: 25; V4: 0

Выполняем вторую итерацию релаксации. Снова просматриваем все рёбра, теперь уже с учётом обновлённых расстояний.

Рёбра, которые дают улучшение:

Из V3 (расстояние 25):

V3 $\rightarrow$ V0: $25+96 = 121 < \infty \rightarrow V0 = 121$
V3 $\rightarrow$ V1: $25+68 = 93 < 99 \rightarrow V1 = 93$
V3 $\rightarrow$ V4: $25+61 = 86 > 0 \rightarrow$ без изменений

Рёбра из других вершин:

Из V0: рёбра есть, но до V0 пока 121 — они могут улучшить V3 и V4, но уже не улучшают.

Из V1: $V1 \rightarrow V3: 93+25=118 > 25 \rightarrow$ без изменений.

Из V4: уже учли, новых улучшений нет.

После 2-й итерации:

V0: 121; V1: 93; V2: 18; V3: 25; V4: 0.

Выполняем третью итерацию релаксации. Пробуем улучшить дальше:

Из V0 (121):

$V0 \rightarrow V3: 121+23=144 > 25 \rightarrow$ нет

$V0 \rightarrow V4: 121+42=163 > 0 \rightarrow$ нет

Из V1 (93):

$V1 \rightarrow V3: 93+25=118 > 25 \rightarrow$ нет

Из V3 (25):

Изменений нет $\rightarrow$ алгоритм завершён.

Выполняем проверку на отрицательный цикл. Делаем ещё один проход по всем рёбрам:

Если какое-то расстояние можно уменьшить, значит есть отрицательный цикл.

$V4 \rightarrow V1: 0+99=99 > 93$

$V4 \rightarrow V2: 0+18=18 = 18$

$V4 \rightarrow V3: 0+25=25 = 25$

$V3 \rightarrow V0: 25+96=121 = 121$

$V3 \rightarrow V1: 25+68=93 = 93$

$V3 \rightarrow V4: 25+61=86 > 0$

$V0 \rightarrow V3: 121+23=144 > 25$

$V0 \rightarrow V4: 121+42=163 > 0$

$V1 \rightarrow V3: 93+25=118 > 25$

Ни одного улучшения нет, значит отрицательных циклов тоже нет.

Выводим результат кратчайших расстояний из V4:

До V0: 121; До V1: 93; До V2: 18; До V3: 25; До V4: 0.

Результаты ручного расчёта полностью совпадают с выводом программы (рис. 11), что подтверждает корректность её работы.

На рисунке представлен подсчет программы.

```
МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 2

=== МАТРИЦА СМЕЖНОСТИ ГРАФА ===

      V0  V1  V2  V3  V4
V0:    0   .   .  23  42
V1:    .   0   .  25   .
V2:    .   .   0   .   .
V3:   96  68   .   0  61
V4:    .  99  18  25   0

МЕНЮ:
1. Создать/загрузить граф
2. Показать текущий граф
3. Выполнить алгоритм Форда-Беллмана
4. Сохранить граф в файл
5. Выйти из программы
Выберите вариант (1-5): 3

Введите номер стартовой вершины (0-4): 4

=== ВЫПОЛНЕНИЕ АЛГОРИТМА ===

=== РЕЗУЛЬТАТЫ ===
Стартовая вершина: 4

До вершины 0: 121
До вершины 1: 93
До вершины 2: 18
До вершины 3: 25
До вершины 4: 0 (стартовая вершина)

Результаты сохранены в файл 'result.txt'
```

Рисунок 11 – Подсчет программы

Заключение

Таким образом, в процессе создания данного курсового проекта была успешно разработана программа, реализующая алгоритм Форда-Беллмана для поиска кратчайших путей из одной вершины во взвешенном ориентированном графе. Программа была разработана в среде Microsoft Visual Studio с использованием языка C.

В ходе работы были получены практические навыки реализации алгоритмов на языке C, освоены приёмы работы с динамическими структурами данных и матрицами смежности для представления графов. Были приобретены умения по реализации операций релаксации рёбер, детектированию циклов отрицательного веса и организации интерактивного взаимодействия с пользователем через консольный интерфейс.

Разработанная программа обеспечивает наглядную демонстрацию работы алгоритма Форда-Беллмана, включая пошаговый вывод промежуточных результатов, что позволяет детально отслеживать процесс поиска кратчайших путей. Результаты ручного расчёта, выполненного для контрольного примера, полностью совпали с результатами работы программы, что подтверждает корректность реализации алгоритма.

Программа обладает расширенной функциональностью по сравнению с базовыми требованиями: поддерживает несколько способов ввода графа (случайная генерация, ручной ввод, загрузка из файла), позволяет работать как с ориентированными, так и с неориентированными графами, корректно обрабатывает рёбра с отрицательными весами и обнаруживает циклы отрицательного веса.

Недостатком разработанной программы является ограничение на максимальный размер графа (7 вершин), что, однако, обусловлено учебным характером проекта и позволяет наглядно демонстрировать работу алгоритма без излишнего усложнения интерфейса.

Программа имеет достаточный функционал для демонстрации работы алгоритма Форда-Беллмана и может быть использована в учебных целях для изучения алгоритмов поиска кратчайших путей во взвешенных графах, а также как основа для разработки более сложных систем анализа сетевых структур.

Список литературы

1. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. – 3-е изд. – М. : Вильямс, 2013. – 1296 с.
2. Ахо А., Хопкрофт Дж., Ульман Дж. Построение и анализ вычислительных алгоритмов. – М. : Мир, 1979. – 536 с.
3. Седжвик Р. Фундаментальные алгоритмы на C++. Части 1-4. Анализ. Структуры данных. Сортировка. Поиск. – К. : ДияСофт, 2001. – 688 с.
4. Вирт Н. Алгоритмы и структуры данных. – М. : Мир, 1989. – 360 с.
5. Кнут Д. Искусство программирования. Том 1. Основные алгоритмы. – 3-е изд. – М. : Вильямс, 2006. – 720 с.

Приложение А.

Листинг программы.

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <time.h>
#include <locale.h>
#include <string.h>

#define INF 99999

int** createMatrix(int size) {
    int** matrix = (int**)malloc(size * sizeof(int*));
    for (int i = 0; i < size; i++) {
        matrix[i] = (int*)calloc(size, sizeof(int));
    }
    return matrix;
}

void freeMatrix(int** matrix, int size) {
    for (int i = 0; i < size; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

void printMatrix(int** matrix, int size) {
    printf("\n=== МАТРИЦА СМЕЖНОСТИ ГРАФА ===\n\n");

    printf("  ");
```

```

    for (int j = 0; j < size; j++) {
        printf(" V%d ", j);
    }
    printf("\n");

    for (int i = 0; i < size; i++) {
        printf("V%d: ", i);
        for (int j = 0; j < size; j++) {
            if (i == j) {
                printf(" 0 ");
            }
            else if (matrix[i][j] == 0) {
                printf(" . ");
            }
            else {
                printf("%3d ", matrix[i][j]);
            }
        }
        printf("\n");
    }
    printf("\n");
}

int** generateRandomGraph(int size, int includeNegative, int isDirected) {
    int** matrix = createMatrix(size);

    srand(time(NULL));

    if (isDirected) {
        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size; j++) {

```

```

        if (i != j) {
            if (rand() % 100 < 50) {
                if (includeNegative) {
                    matrix[i][j] = (rand() % 121) - 20;
                }
                else {
                    matrix[i][j] = rand() % 100 + 1;
                }
            }
        }
    }
}

else {
    for (int i = 0; i < size; i++) {
        for (int j = i + 1; j < size; j++) {
            if (rand() % 100 < 50) {
                int weight;
                if (includeNegative) {
                    weight = (rand() % 121) - 20;
                }
                else {
                    weight = rand() % 100 + 1;
                }
                matrix[i][j] = weight;
                matrix[j][i] = weight;
            }
        }
    }
}
}

```

```

    return matrix;
}

int** inputGraphFromKeyboard(int size, int isDirected) {
    int** matrix = createMatrix(size);

    printf("\nВведите веса ребер (0 - если ребра нет):\n");

    if (isDirected) {
        printf("(Ориентированный граф - ребра могут быть направленными)\n");
        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size; j++) {
                if (i == j) {
                    matrix[i][j] = 0;
                    continue;
                }

                printf("Вес ребра V%d -> V%d: ", i, j);
                scanf("%d", &matrix[i][j]);
            }
        }
    }
    else {
        printf("(Неориентированный граф - ребра симметричные)\n");
        for (int i = 0; i < size; i++) {
            for (int j = i; j < size; j++) {
                if (i == j) {
                    matrix[i][j] = 0;
                    continue;
                }
            }
        }
    }
}

```



```

        printf("Вес ребра V%d <-> V%d: ", i, j);
        int weight;
        scanf("%d", &weight);
        matrix[i][j] = weight;
        matrix[j][i] = weight;
    }
}

return matrix;
}

int** loadGraphFromFile(const char* filename, int* size) {
    FILE* file = fopen(filename, "r");
    if (!file) {
        printf("\nОшибка: не удалось открыть файл '%s'\n", filename);
        return NULL;
    }

    if (fscanf(file, "%d", size) != 1) {
        printf("Ошибка: неверный формат файла\n");
        fclose(file);
        return NULL;
    }

    if (*size <= 0) {
        printf("Ошибка: некорректный размер графа\n");
        fclose(file);
        return NULL;
    }
}

```

```

int** matrix = createMatrix(*size);

for (int i = 0; i < *size; i++) {
    for (int j = 0; j < *size; j++) {
        if (fscanf(file, "%d", &matrix[i][j]) != 1) {
            printf("Ошибка: недостаточно данных в файле\n");
            freeMatrix(matrix, *size);
            fclose(file);
            return NULL;
        }
    }
}

fclose(file);
printf("\nГраф успешно загружен из файла '%s'\n", filename);
return matrix;
}

void saveGraphToFile(int** matrix, int size, const char* filename) {
    FILE* file = fopen(filename, "w");
    if (!file) {
        printf("\nОшибка: не удалось создать файл '%s'\n", filename);
        return;
    }

    fprintf(file, "%d\n", size);
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            fprintf(file, "%d ", matrix[i][j]);
        }
        fprintf(file, "\n");
    }
}

```

```

    }

    fclose(file);
    printf("\nГраф успешно сохранен в файл '%s'\n", filename);
}

void bellmanFord(int** matrix, int size, int start) {
    int* distance = (int*)malloc(size * sizeof(int));
    int hasNegativeCycle = 0;

    for (int i = 0; i < size; i++) {
        distance[i] = INF;
    }
    distance[start] = 0;

    printf("\n=== ВЫПОЛНЕНИЕ АЛГОРИТМА ===\n");

    for (int iteration = 1; iteration <= size - 1; iteration++) {
        int changes = 0;

        for (int from = 0; from < size; from++) {
            for (int to = 0; to < size; to++) {
                if (matrix[from][to] != 0) {
                    if (distance[from] != INF &&
                        distance[from] + matrix[from][to] < distance[to]) {
                        distance[to] = distance[from] + matrix[from][to];
                        changes = 1;
                    }
                }
            }
        }
    }
}

```

```

    if (changes == 0) {
        break;
    }
}

for (int from = 0; from < size; from++) {
    for (int to = 0; to < size; to++) {
        if (matrix[from][to] != 0) {
            if (distance[from] != INF &&
                distance[from] + matrix[from][to] < distance[to]) {
                hasNegativeCycle = 1;
                break;
            }
        }
    }
    if (hasNegativeCycle) break;
}

```

```

printf("\n=== РЕЗУЛЬТАТЫ ===\n");
printf("Стартовая вершина: %d\n\n", start);

```

```

if (hasNegativeCycle) {
    printf("Граф содержит отрицательный цикл!\n");
    printf("Кратчайшие пути не существуют!\n\n");
}

```

```

for (int i = 0; i < size; i++) {
    printf("До вершины %d: ", i);
    if (distance[i] == INF) {
        printf("нет пути\n");
    }
}

```

```

    }
    else if (i == start) {
        printf("0 (стартовая вершина)\n");
    }
    else {
        printf("%d\n", distance[i]);
    }
}

FILE* file = fopen("result.txt", "w");
if (file) {
    fprintf(file, "РЕЗУЛЬТАТЫ АЛГОРИТМА ФОРДА-БЕЛЛМАНА\n");
    fprintf(file, "Стартовая вершина: %d\n\n", start);

    if (hasNegativeCycle) {
        fprintf(file, "ВНИМАНИЕ: Граф содержит отрицательный цикл!\n");
        fprintf(file, "Кратчайшие пути не существуют!\n\n");
    }

    for (int i = 0; i < size; i++) {
        if (distance[i] == INF) {
            fprintf(file, "Вершина %d: недостижима\n", i);
        }
        else if (i == start) {
            fprintf(file, "Вершина %d (стартовая): 0\n", i);
        }
        else {
            fprintf(file, "Вершина %d: %d\n", i, distance[i]);
        }
    }
}

```

```

        fclose(file);
        printf("\nРезультаты сохранены в файл 'result.txt'\n");
    }

    free(distance);
}

int getNumber(const char* prompt, int min, int max) {
    int value;
    while (1) {
        printf("%s", prompt);
        if (scanf("%d", &value) == 1 && value >= min && value <= max) {
            return value;
        }
        printf("Ошибка! Введите число от %d до %d\n", min, max);
        while (getchar() != '\n');
    }
}

void clearInputBuffer() {
    while (getchar() != '\n');
}

int getGraphType() {
    int choice;
    while (1) {
        printf("\nВыберите тип графа:\n");
        printf("1. Ориентированный граф (направленные ребра)\n");
        printf("2. Неориентированный граф (симметричные ребра)\n");
        printf("Выберите вариант (1-2): ");
    }
}

```

```

        if (scanf("%d", &choice) != 1 || (choice != 1 && choice != 2)) {
            printf("Ошибка! Пожалуйста, выберите 1 или 2\n");
            clearInputBuffer();
            continue;
        }
        clearInputBuffer();
        return choice;
    }
}

int main() {
    setlocale(LC_ALL, "Russian");

    int** matrix = NULL;
    int size = 0;
    int choice;

    printf("    КУРСОВАЯ РАБОТА\n");
    printf(" по теме: АЛГОРИТМ ФОРДА-БЕЛЛМАНА\n");
    printf(" Выполнил студент группы 24BBB1:\n");
    printf("                Хмельков М.А.\n");
    printf("=====\\n\\n");

    while (1) {
        printf("\\nМЕНЮ:\\n");
        printf("1. Создать/загрузить граф\\n");
        printf("2. Показать текущий граф\\n");
        printf("3. Выполнить алгоритм Форда-Беллмана\\n");
        printf("4. Сохранить граф в файл\\n");
        printf("5. Выйти из программы\\n");
        printf("Выберите вариант (1-5): ");
    }
}

```

```

if (scanf("%d", &choice) != 1) {
    printf("Ошибка ввода!\n");
    clearInputBuffer();
    continue;
}
clearInputBuffer();

switch (choice) {
case 1: {
    printf("\n=== ВЫБЕРИТЕ СПОСОБ СОЗДАНИЯ ГРАФА ===\n");
    printf("1. Автоматическая генерация (случайный граф)\n");
    printf("2. Ручной ввод с клавиатуры\n");
    printf("3. Загрузка из файла\n");
    printf("4. Вернуться в главное меню\n");
    printf("Выберите вариант (1-4): ");

    int graphChoice;
    scanf("%d", &graphChoice);
    clearInputBuffer();

    if (matrix != NULL) {
        freeMatrix(matrix, size);
        matrix = NULL;
    }

    switch (graphChoice) {
case 1: {
        size = getNumber("\nВведите количество вершин (2-7): ", 2, 7);

        int graphType = getGraphType();

```



```

int isDirected = (graphType == 1);

printf("\nВключить отрицательные веса?\n");
printf("1. Да\n");
printf("2. Нет\n");
printf("Выберите (1-2): ");
int weightChoice;
scanf("%d", &weightChoice);
clearInputBuffer();

matrix = generateRandomGraph(size, weightChoice == 1, isDirected);
printf("\nСлучайный граф успешно создан!\n");
if (isDirected) {
    printf("Тип: ориентированный граф\n");
}
else {
    printf("Тип: неориентированный граф\n");
}
break;
}

case 2: {
    size = getNumber("\nВведите количество вершин (2-7): ", 2, 7);

    int graphType = getGraphType();
    int isDirected = (graphType == 1);

    matrix = inputGraphFromKeyboard(size, isDirected);
    printf("\nГраф успешно введен!\n");
    if (isDirected) {
        printf("Тип: ориентированный граф\n");
    }
}

```

```

    }
    else {
        printf("Тип: неориентированный граф\n");
    }
    break;
}

case 3: {
    char filename[100];
    printf("\nВведите имя файла: ");
    fgets(filename, sizeof(filename), stdin);
    filename[strcspn(filename, "\n")] = 0;

    matrix = loadGraphFromFile(filename, &size);
    break;
}

case 4:
    break;

default:
    printf("Неверный выбор!\n");
}
break;
}

case 2: {
    if (matrix == NULL) {
        printf("\nГраф не создан! Сначала создайте или загрузите граф.\n");
    }
    else {

```

```

        printMatrix(matrix, size);
    }
    break;
}

case 3: {
    if (matrix == NULL) {
        printf("\nГраф не создан! Сначала создайте или загрузите граф.\n");
    }
    else {
        printf("\nВведите номер стартовой вершины (0-%d): ", size - 1);
        int start;
        scanf("%d", &start);
        clearInputBuffer();

        if (start < 0 || start >= size) {
            printf("Ошибка! Стартовая вершина должна быть от 0 до %d\n", size -
1);
        }
        else {
            bellmanFord(matrix, size, start);
        }
    }
    break;
}

case 4: {
    if (matrix == NULL) {
        printf("\nГраф не создан! Нечего сохранять.\n");
    }
    else {
        char filename[100];

```

```

        printf("\nВведите имя файла для сохранения (по умолчанию graph.txt): ");
        fgets(filename, sizeof(filename), stdin);
        filename[strcspn(filename, "\n")] = 0;

        if (strlen(filename) == 0) {
            strcpy(filename, "graph.txt");
        }

        saveGraphToFile(matrix, size, filename);
    }
    break;
}

case 5: {
    printf("\nПрограмма завершена. (0__0)\n");

    if (matrix != NULL) {
        freeMatrix(matrix, size);
    }
    return 0;
}

default:
    printf("Неверный выбор! Пожалуйста, выберите 1-5.\n");
}

return 0;
}

```